College of Computer
كلية الحاسب
Saudi Arabia
Qassim University
جامعة القصيم

Algorithm design and analysis project

CS315 – group: 6671

Task scheduling and time management

Name:	ID:
Miqdad Osama	451115067

Instructor: Dr. Nouf Mohammed Al-harbi

• Contacts:

P1 **Overview:**

1.1 introduction 3

1.2 problem description 3

P2 **Theoretical Analysis:**

2.1 Navi algorithm 4

- 2.1.1 Pseudocode 4
- 2.1.2 Analysis 5
- 2.1.3 Example 10

2.2 Optimized Algorithm 12

- 2.2.1 pseudocode 13
- 2.2.2 Analysis 16
- 2.2.3 Example 22

2.3 Comparison 24

P3 **Empirical Analysis**

- 3.1 Navie Algorithm Implementation 25
- 3.2 Optimized Algorithm Implementation 26
- 3.3 Performance Analysis 29

P4 **Conclusion**

- 4.1 Comparison 30
- 4.2 Final Thoughts 30

Overview

1.1 introduction:

This project discusses one of the most pressing problems we face every day in our lives: **scheduling and time management.**

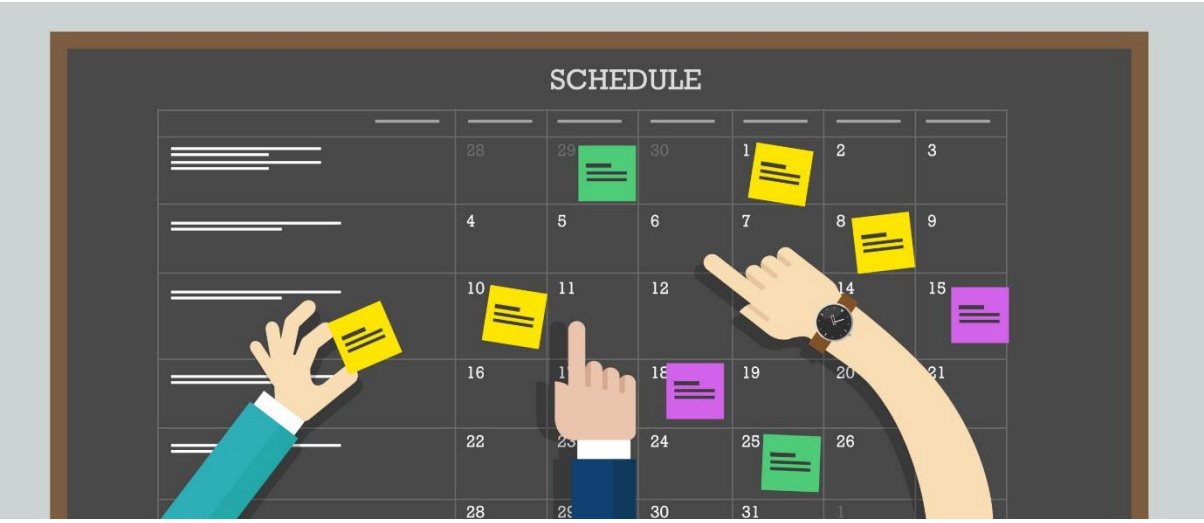
Scheduling and time management are one of the most important skills you have to learn, but with large schedules it is difficult to track each task individually and check if it fits perfectly in this time without any conflict.

In this project, we aim to make life easier by developing a system that simplifies tracking and makes adding new tasks easier without any headache.

1.2 problem description :

In our daily life, it is often difficult to organize the schedules effectively. Because of this, people waste a lot of time trying to organize and add a new task, especially during busy days. Also, because you cannot make your schedule organized, it will affect efficiency and reduce performance and productivity. It becomes difficult to track which task is important and you need to perform it first.

It's not affecting your performance in your job only, but also your personal life, you don't have time for your family, poor meal planning, and insufficient sleep, all of which are just as important as our job.



Theoretical Analysis

2.1 Navie algorithm:

Description (Rationale):

The naïve algorithm of scheduling and time management works by comparing the start and end times, as well as the weight of each task, before placing it in the schedule.

The algorithm receives the input from the user and performs the process as follows:

1. Receiving the day, task name, start time, end time, and the weight of the task.

2. The weight of the task will be determined by three letters:

- M = Very important
- P = Important but can be postponed
- O = Open time (less important)

3. Compare each task's time and the weight with others to determine the most appropriate time to place it.

The following pseudocode describes step-by-step how we can perform the naïve algorithm:

2.1.1 Pseudocode:

Algorithm 1: Naive Scheduling and Time Management Algorithm

```
Inputs: day: First input
        taskName: Second input
        startTime: Third input
        endTime: Fourth input
        weight: Fifth input
Output: fully organized schedule

Function enter_New_Task(day, taskName , startTime , endTime , weight ):

    name ← convert taskName to lowercase
    ST ← startTime
    ET ← endTime
    W ← convert weight to uppercase
```

```
Open the task file (myTask)

 If the file is not open then
        Display "Error: the selected file is not opened"
        Return
 end if

Read the existing tasks from the file line by line

        state ← FALSE

        For each existing task in the file do

                a. Extract the fields (day , name , ST , ET , W)

                b. Check for time overlap :

                        if (day == old_task_day)AND
                        ((ST > old_task_ST AND ST < old_task_ET)
                        OR (ET > old_task_ST AND ET <old_task_ET)
                        OR (ST ≤ old_task_ST AND ET ≥ old_task_ET))
                        then
                                state ← TRUE
                                break

                      End if
        End for

 if state is TRUE then
        Display "Sorry the new task is overlap with another task"
else
        append the new task to the file
        Display "The new task was added successfully"
End if

Close the file

end function
```

It is named SCIN algorithm (S stand for search, C for comparison, and IN for insertion)

2.1.2 Analysis:

	#Opreation
1. name ← convert taskName to lowercase	1
2. ST ← startTime	1
3. ET ← endTime	1
4. W ← convert weight to uppercase	1
5. **Open the task file** (myTask)	1
6. If the file is not open **then**	2
i. **Display** "Error: the selected file is not opened"	1
ii. **Return**	1
end if	
7. Read the existing tasks from the file line by line	**n**
i. state ← **FALSE**	1
ii. **For each existing task in the file do**	**n+1**
b. Extract the fields (day , name , ST , ET , W)	**1**
c. Check for time overlap :	
if (day == old_task_day)**AND**	
((ST > old_task_ST AND ST < old_task_ET)	
OR (ET > old_task_ST AND ET <old_task_ET)	
OR (ST ≤ old_task_ST AND ET ≥ old_task_ET))	
then	**n**
a. state ← **TRUE**	1
b. break	1
End if	
End for	
8. if state is TRUE **then**	1
i. **Display** "Sorry the new task overlaps with another task"	1
9. else	
i. **append** the new task to the file	1
ii. **Display** "The new task was added successfully"	1
End if	
Close file	1

Time complexity of the Naive Algorithm:

My algorithm processes the input in several steps: opening the file containing the tasks, checking whether it is opened successfully, and then reading the file line by line, extracting the fields and storing them in variables for comparison purposes. After extracting the fields, the algorithm detects any overlap. Below is the time complexity of each operation:

Opening and checking the file:

The function will open the file and check if it is opened successfully and all of that will take constant time.

Time complexity is O(1).

Reading the file:

The function will perform the reading operation for file, and it takes only a Linear time to perform it.

Time complexity: O(n)

Extracting the fields:

During reading the file, the function extracts the fields and saves them into variables: the day you want to perform the task in the var(day), the name of the task in the var(name), starting time in var(ST), ending time in var(ET), and the weight in var(W).

This operation also will take linear time relative to the number of tasks.

Time complexity: O(n)

Detecting overlap:

After extracting the fields, the condition will check if there are any overlaps with old data in the file line by line. For example, if the task starts before any old task in the file but ends in the middle of this task, causing an overlap and so on.

Because it is checked line by line, the time complexity is O(n).

Insertion into the file operation:

This is the last step; after checking for overlaps, the condition will return *TRUE* or *FALSE.* If it is *TRUE*, it will not add the new task to the schedule and display the message, but if it returns *FALSE* that means that no overlaps are detected in the schedule, so it will add the new task.

This is performed once (constant time), so the time complexity is O(1).

Overall time complexity:

The time complexity of the naive algorithm is determined by the dominant term in the analysis.

The time complexity for all terms is: O(n).

Space complexity analysis of the Naive Algorithm:

The space required by any algorithm is as important as its time complexity; calculating the space can explain how much space it needs before executing the actual program, and this can help in optimizing the algorithm in the future for better efficiency. Below is the calculation of space for every operation:

Variables:

The variables used in this algorithm, such as ST, name, and state, require constant memory that does not depend on the number of tasks or the size of the file:

Space complexity is O(1)

File:

Since the algorithm does not store the tasks in arrays or vectors and processes the file line by line using temporary variables, it also requires only constant memory.

Space complexity is also O(1)

The reason why this algorithm is efficient in space complexity is that it reads and compares the data line by line, using only temporary variables. After this process, the new task is inserted into the same file at the last line without any rearrangement.

Overall Space Complexity: O(1)

Algorithm cases:

The time complexity for all cases can show the strengths and weaknesses of the algorithm with different sizes of data.

Best case:

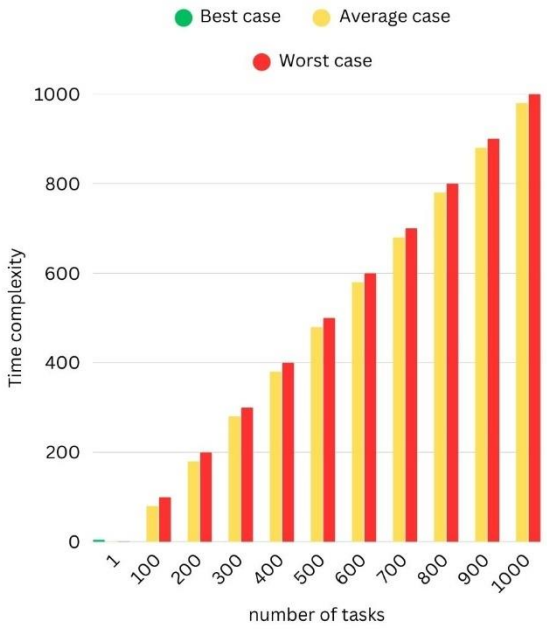
When the file is empty, there is no need to compare. The complexity is only **constant time O(1).**

Average case:

When the file has small amounts of tasks and the comparisons are required, the complexity is **linear time O(n).**

Worst case:

The worst case is very important to us because it shows how the algorithm works under extreme conditions. In this algorithm, the worst case occurs when the data is large, and the entire file must be compared. The complexity is **linear time O(n).**



2.1.3 Example for Naive Algorithm

Input:

- day: 3
- taskName: "Math quiz "
- startTime: "11:00 "
- endTime: "12:00 "
- weight: "M"

step 1: opening the file and checking if it is opened successfully

the data from file:

```
1 Physics exam 10:00 11:00 M
2 meeting 1:00 1:30 O
3 Homework assignment 4:00 5:00 M
```

step 2: Reading the file line by line and extracting the fields

First task:

- day = 1
- name = Physics exam
- ST= 10:00
- ET = 11:00
- W = M

Step 3: checking the overlap

The day of the new task is not the same as the day of existing task.

Since the days are not equal, the condition will not be performed.

State will remain false.

Go back to Step 2.

Reading the file:

Second task:

- day = 2
- name = meeting

- ST= 1:00
- ET = 1:30
- W = O

Checking for overlap:

Since the day is not equal, the condition will not be performed.

State will remain false.

Go back to Step 2 again.

Reading the file:

Third task:

- day = 3
- name = Homework
- ST= 4:00
- ET = 5:00
- W = M

The day is equal, but the starting and ending times are different, so the condition will not be performed.

State will remain false.

Step 4: insertion operation

Since the state remains false for all tasks, we can insert the new task with any problems.

The schedule after update:

1 Physics exam 10:00 11:00 M 2 meeting 1:00 1:30 O 3 Homework assignment 4:00 5:00 M 3 Math quiz 11:00 12:00 M

2.2 Optimized Algorithm

Description (Rationale):

The optimized algorithm for *SCIN* is designed to improve efficiency and performance, allowing it to work faster under extreme cases and deal with large schedules. This algorithm minimizes the dependency on reading from the hard disk, as this makes the operation extremely slow. Also, unlike the naive algorithm where the insertion operation places the new task at the last line in the file, we developed a rearrangement method to make the schedule organized. The algorithm proceeds as follows:

1. Receiving the input from the user: day, starting and ending times, and checking whether the times are entered correctly (such as checking the start time is less than the end time), as well as the weight of the new task.

2. Load data from the file and store it in an array or vector.

3. Insertion operation: insert the new task, after checking the overlap, in the correct place.

4. The last step is to save the data in the file.

2.2.1 Pseudocode:

```
Algorithm 2: Optimized Scheduling and Time Management Algorithm
Inputs: day: First input
        taskName: Second input
        startTime: Third input
        endTime: Fourth input
        weight: Fifth input
Output: fully organized schedule

Function tasksFile():

    Open the task file (myTask)

    If the file is not open then
        Display "Error: the selected file is not opened"
        return empty list
    End if

    initialize empty array tasksList

    Read the existing tasks from the file line by line

        For each existing task in the file do

            Extract the fields (day, name, ST, ET, W)
            task ← Task(day, name, ST, ET, W)
            append task to tasksList
        End for

    Close file

    return tasksList
End function


FileOrder(tasksList):

    n ← length (tasksList)
    if n ⩽ 1 then
        return tasksList
    End if
```

```
    mid ← n / 2

    left ← FileOrder (tasksList [1 : mid])
    Right ← FileOrder(tasksList [mid + 1: n])

    return Merge (left, Right)
End function

Function Merge(left, Right):

        initialize empty array sortedList
        i ← 1
        j ← 1

       while i ⩽ length(left) AND j ⩽ length(right) do
                if left[i].day < right[j].day OR
                  (left[i]. day = right[j].day AND left[i].ST ⩽ right[j].ST)
                    then
                          append left[i] to sortedList
                          i ← i + 1

                else
                        append right[j] to sortedList
                          j ← j + 1

               end if
        end while

        while i ⩽ length(left) do

                  append left[i] to sortedList
                   i ← i + 1
        end while

        while j ⩽ length(right) do

                  append right[j] to sortedList
                   j ← j + 1
        end while

        return sortedList
End function
```

```
Function enter_New_Task(day, taskName, startTime, endTime, weight):

    name ← convert taskName to lowercase
    ST ← startTime
    ET ← endTime
    W ← convert weight to uppercase

    if ST ⩾ ET then
       Display "Error: The starting time must be less than ending time "
       return

    tasksList ← tasksFile()
    state ← FALSE

    For each task in the array tasksList do
          if (day == task.day) AND
             ((ST > task.ST AND ST < task.ET)
              OR (ET > task.ST AND ET <task.ET)
              OR (ST ≤ task.ST AND ET ≥ task.ET))
              then
                     state ← TRUE
                     break
          End if
    End for

     if state is TRUE then
            Display "Sorry the new task overlaps with another task"

    else
              newTask ← Task(day, name, ST, ET ,W)

              append the newTask to the tasksList

              tasksList ← FileOrder(tasksList)

              write tasksList back to file(myTask)

              Display "The new task was added successfully"
    End if

End function
```

SCIN+ Optimized Algorithm

2.2.2 Analysis:

```
                                                        #Opreation
Function tasksFile():

    Open the task file (myTask)                          1

    If the file is not open then
        Display "Error: the selected file is not opened"  1
        return empty list                                 1
    End if

    initialize empty array tasksList                     1

    Read the existing tasks from the file line by line

        For each existing task in the file do            n+1

            Extract the fields (day, name, ST, ET, W)    n
            task ← Task(day, name, ST, ET, W)            n
            append task to tasksList                     n
        End for

    Close file                                           1

    return tasksList                                     1
End function

FileOrder(tasksList):                                    T(n)

    n ← length (tasksList)                               1
    if n ⩽ 1 then                                        1
        return tasksList                                 1
    End if

    mid ← n / 2                                          1

    left ← FileOrder (tasksList [1 : mid])               T(n/2)
    Right ← FileOrder(tasksList [mid + 1: n])            T(n/2)

    return Merge (left, Right)                           n
End function
```

```
Function Merge(left, Right):

        initialize empty array sortedList                                  1
        i ← 1                                                              1
        j ← 1

       while i ⩽ length(left) AND j ⩽ length(right) do                     n+1
              if left[i].day < right[j].day OR
                (left[i]. day = right[j].day AND left[i].ST ⩽ right[j].ST)
                  then                                                     n
                        append left[i] to sortedList                       n
                        i ← i + 1                                          n

              else
                      append right[j] to sortedList                        n
                        j ← j + 1                                          n

             end if
       end while

       while i ⩽ length(left) do                                           n+1

                 append left[i] to sortedList                              n
                  i ← i + 1                                                n
       end while

       while j ⩽ length(right) do                                          n+1

                 append right[j] to sortedList                             n
                  j ← j + 1                                                n
       end while

       return sortedList                                                   1
End function

Function enter_New_Task(day, taskName, startTime, endTime, weight):

     name ← convert taskName to lowercase                                  1
     ST ← startTime                                                        1
     ET ← endTime                                                          1
     W ← convert weight to uppercase                                       1
```

```
if ST ⩾ ET then
   Display "Error: The starting time must be less than ending time "     1
   return                                                                1

tasksList ← tasksFile()                                                  1
state ← FALSE                                                            1

For each task in the array tasksList do                                  n+1
      if (day == task.day) AND                                           n
         ((ST > task.ST AND ST < task.ET)
          OR (ET > task.ST AND ET <task.ET)
          OR (ST ≤ task.ST AND ET ≥ task.ET))
          then
                state ← TRUE                                             n
                break                                                    1
      End if
End for

 if state is TRUE then
        Display "Sorry the new task overlaps with another task"          1

else
         newTask ← Task(day, name, ST, ET, W)                            1

         append the newTask to the tasksList                             1

         tasksList ← FileOrder(tasksList)                                T(n)

         write tasksList back to file(myTask)                            n

         Display "The new task was added successfully"                   1
End if

End function
```

Time complexity of the Optimized Algorithm:

The Optimized Algorithm begins by opening the file and reading all existing tasks into an array. It then checks the new task for any overlaps with existing tasks and determines whether it can be inserted. If valid, the new task is added to the list, after which the sorting function is called to arrange all tasks in correct order. Finally, the updated task list is written back to the file.

Opening and reading the file:

The function opens the file and checks if it was opened successfully, which takes constant time. Reading the file takes linear time to perform it.

Time complexity: O(n)

Extracting the fields:

While reading the file, the function extracts the fields, and saves them into variables, and inserts them into an array.

This operation also will take linear time relative to the number of tasks.

Time complexity: O(n)

Checking for overlap:

After reading the file and inserting all tasks into array, the function checks whether the new task overlaps with any existing tasks. If the task is valid, the state is updated accordingly.

This operation also will take linear time relative to the number of tasks.

Time complexity: O(n)

Insertion, sorting, and writing back to file:

After checking for overlapping and updating the state, the new task is inserted into the array. Next, the sorting function is called to arrange all tasks in correct order. Finally, the updated task list is written back to file.

Time complexity: O(n log n)

Overall time complexity:

The Overall time complexity of the optimized algorithm is determined by the most time-consuming step, which is the sorting process. Therefore, the time complexity for all terms is O(n log n).

Space complexity analysis of the Optimized Algorithm:

The space required by any algorithm is as important as its time complexity; calculating the space can explain how much space it needs before executing the actual program. Below is the calculation of space for every operation:

Variables:

The variables used in this algorithm, such as ST, name, and state, require constant memory that does not depend on the number of tasks or the size of the file:

Space complexity is O(1).

Array and Sorting function:

The algorithm also stores all tasks in auxiliary array and additional memory during merge sort process. which increases the memory requirements relative to number of tasks.

Space complexity is O(n).

Overall Space Complexity: O(n)

Algorithm cases:

The time complexity for all cases can show the strengths and weaknesses of the algorithm with different sizes of data.

Best case:

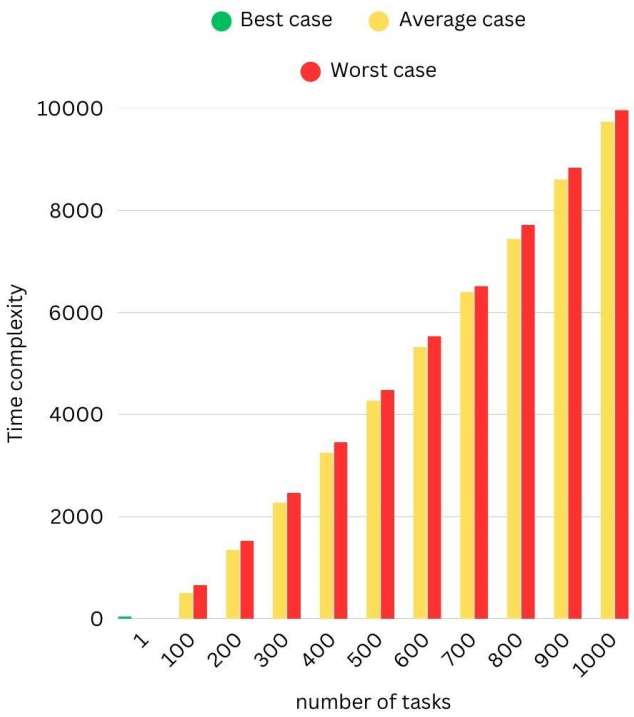
When the task file is empty (n = 0), the algorithm performs only constant operations. Time complexity is **O(1).**

Average case:

Under typical conditions, the algorithm reads the tasks, checks for overlap, and sorts the data by using merge sort. Time complexity is **O(n log n).**

Worst case:

The worst case is very important to us because it shows how the algorithm works under extreme conditions. In this algorithm, the worst case occurs when the data is large. The complexity is **O(n log n).**



2.2.3 Example for Optimized Algorithm

Input :

- day: 3
- taskName: "Math quiz "
- startTime: "11:00 "
- endTime: "12:00 "
- weight: "M"

step 1: opening the file and checking if it is opened successfully

the data from file:

```
1 Physics exam 10:00 11:00 M
2 meeting 1:00 1:30 O
3 Homework assignment 4:00 5:00 M
```

step 2: Reading the file line by line and extracting the fields

First task:

- day = 1
- name = Physics exam
- ST= 10:00
- ET = 11:00
- W = M

Step 3: Store it in array by index:

0 for the first task

Second task:

- day = 2
- name = meeting
- ST= 1:00
- ET = 1:30
- W = O

0 for the first task	1 for the second task

Third task:

- day = 3
- name = Homework
- ST= 4:00
- ET = 5:00
- W = M

0 for the first task	1 for the second task	2 for third task

<u>Step 4: checking the overlap:</u>

There are no overlaps with existing tasks; therefore, we can insert the new task in the array.

0 for the first task	1 for the second task	2 for third task	3 for new task

<u>Step 5: calling sorting function:</u>

0 for the first task	1 for the second task	2 for new task	3 for third task

<u>Step 6: Writing back to file:</u>

The schedule after update:

1 Physics exam 10:00 11:00 M 2 meeting 1:00 1:30 O 3 Math quiz 11:00 12:00 M 3 Homework assignment 4:00 5:00 M

2.3 Theoretical analysis: Comparison

The Naive algorithm appears to be faster than optimized algorithms in terms asymptotic time complexity (O(n) vs O(n log n)) and theoretical requires less memory space (O(1) vs O(n)). However, in reality, the number is not enough to show the real performance.

The Naive algorithm, despite its linear time complexity (O(n)), is often slow because its operations are heavily dependent on accessing the **hard drive**. In contrast, the optimized algorithm utilizes main memory (RAM) to perform core processing (like sorting), which makes them significantly faster in terms of data manipulation and loading speed after initial file read.

Secondly, time complexity of the optimized algorithms is (n log n), which is mathematical slower than naive algorithm. However, here is the crucial distinction: while naive algorithm is faster with a small number of tasks, with large amounts of data, the constant, slow instruction with hard drive makes it inefficient and resources intensive. This inefficiency can lead to performance degradation or system instability. Conversely, optimized algorithm is very stable and reliable when handling large amounts of data.

naive algorithm:

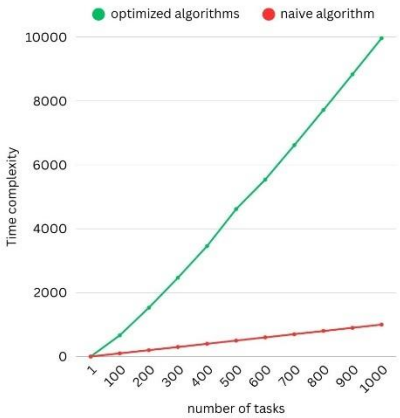
The advantage of the naive algorithm is that it is very simple to implement.

The disadvantage is unstable with a large amount of data.

optimized algorithms:

The advantage is that it is very stable and reliable with a large amount of data.

The disadvantage is slow with small amounts of data and slightly more complex to implement compared to the naive algorithm.



Empirical Analysis

3.1 Naive Algorithm Implementation:

Here is the implementation of the naive algorithm in C++.

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <algorithm>

using namespace std;

// Convert string to lowercase
string toLowerCase(string str) {
    transform(str.begin(), str.end(), str.begin(), ::tolower);
    return str;
}

// Convert string to uppercase
string toUpperCase(string str) {
    transform(str.begin(), str.end(), str.begin(), ::toupper);
    return str;
}

// Function to enter a new task
void enter_New_Task(string day, string taskName, string startTime, string endTime, string weight) {
    string name = toLowerCase(taskName);
    string ST = startTime;
    string ET = endTime;
    string W = toUpperCase(weight);

    ifstream inFile("myTask.txt");
    if (!inFile.is_open()) {
        cout << "Error: the file could not be opened" << endl;
        return;
    }

    string line;
    bool overlap = false;

    // Check for overlapping tasks
    while (getline(inFile, line)) {
        string old_day, old_name, old_ST, old_ET, old_W;
        stringstream ss(line);
        getline(ss, old_day, ',');
        getline(ss, old_name, ',');
        getline(ss, old_ST, ',');
        getline(ss, old_ET, ',');
        getline(ss, old_W, ',');

        if (day == old_day &&
            ((ST > old_ST && ST < old_ET) ||
             (ET > old_ST && ET < old_ET) ||
             (ST <= old_ST && ET >= old_ET))) {
            overlap = true;
            break;
        }
    }
    inFile.close();
```

```cpp
    inFile.close();

    if (overlap) {
        cout << "Sorry, the new task overlaps with another task" << endl;
    } else {
        ofstream outFile("myTask.txt", ios::app);
        if (!outFile.is_open()) {
            cout << "Error: cannot open file to write" << endl;
            return;
        }
        outFile << day << "," << name << "," << ST << "," << ET << "," << W << endl;
        outFile.close();
        cout << "The new task was added successfully" << endl;
    }
}
```

3.2 Optimized Algorithm Implementation

Below is the implementation of the optimized algorithm.

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;

// Task structure
struct Task {
    string day;
    string name;
    string ST;
    string ET;
    string W;

    Task() {}
    Task(string d, string n, string st, string et, string w)
        : day(d), name(n), ST(st), ET(et), W(w) {}
};

// Convert string to lowercase
string toLowerCase(string str) {
    transform(str.begin(), str.end(), str.begin(), ::tolower);
    return str;
}

// Convert string to uppercase
string toUpperCase(string str) {
    transform(str.begin(), str.end(), str.begin(), ::toupper);
    return str;
}

// Read tasks from file into a vector
vector<Task> tasksFile() {
    vector<Task> tasksList;
    ifstream inFile("myTask.txt");
    if (!inFile.is_open()) {
        cout << "Error: the file could not be opened" << endl;
        return tasksList; // return empty list
    }

    string line;
    while (getline(inFile, line)) {
        string day, name, ST, ET, W;
        stringstream ss(line);
        getline(ss, day, ',');
        getline(ss, name, ',');
        getline(ss, ST, ',');
        getline(ss, ET, ',');
        getline(ss, W, ',');
        tasksList.push_back(Task(day, name, ST, ET, W));
    }

    inFile.close();
    return tasksList;
}
```

```
// Merge function for merge sort
vector<Task> Merge(const vector<Task>& left, const vector<Task>& right) {
    vector<Task> sortedList;
    size_t i = 0, j = 0;

    while (i < left.size() && j < right.size()) {
        if (left[i].day < right[j].day ||
            (left[i].day == right[j].day && left[i].ST <= right[j].ST)) {
            sortedList.push_back(left[i]);
            i++;
        } else {
            sortedList.push_back(right[j]);
            j++;
        }
    }

    while (i < left.size()) {
        sortedList.push_back(left[i]);
        i++;
    }

    while (j < right.size()) {
        sortedList.push_back(right[j]);
        j++;
    }

    return sortedList;
}

// Merge sort to order tasks by day and start time
vector<Task> FileOrder(const vector<Task>& tasksList) {
    if (tasksList.size() <= 1)
        return tasksList;

    size_t mid = tasksList.size() / 2;
    vector<Task> left(tasksList.begin(), tasksList.begin() + mid);
    vector<Task> right(tasksList.begin() + mid, tasksList.end());

    left = FileOrder(left);
    right = FileOrder(right);

    return Merge(left, right);
}

// Write tasks back to file
void writeTasksToFile(const vector<Task>& tasksList) {
    ofstream outFile("myTask.txt");
    if (!outFile.is_open()) {
        cout << "Error: cannot open file to write" << endl;
        return;
    }

    for (const auto& task : tasksList) {
        outFile << task.day << "," << task.name << "," << task.ST << ","
                << task.ET << "," << task.W << endl;
    }

    outFile.close();
}
```

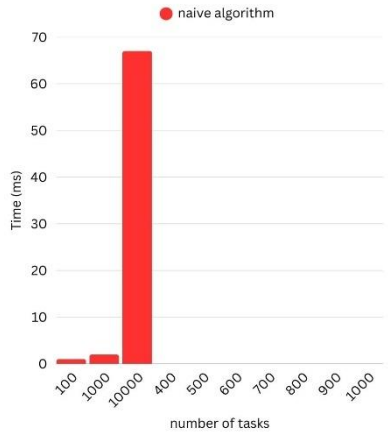
```
// Add a new task
void enter_New_Task(string day, string taskName, string startTime, string endTime, string weight) {
    string name = toLowerCase(taskName);
    string ST = startTime;
    string ET = endTime;
    string W = toUpperCase(weight);

    if (ST >= ET) {
        cout << "Error: The starting time must be less than ending time" << endl;
        return;
    }

    vector<Task> tasksList = tasksFile();
    bool overlap = false;

    // Check for overlaps
    for (const auto& task : tasksList) {
        if (day == task.day &&
            ((ST > task.ST && ST < task.ET) ||
             (ET > task.ST && ET < task.ET) ||
             (ST <= task.ST && ET >= task.ET))) {
            overlap = true;
            break;
        }
    }

    if (overlap) {
        cout << "Sorry, the new task overlaps with another task" << endl;
    } else {
        tasksList.push_back(Task(day, name, ST, ET, W));
        tasksList = FileOrder(tasksList);  // Sort tasks
        writeTasksToFile(tasksList);
        cout << "The new task was added successfully" << endl;
    }
}
```

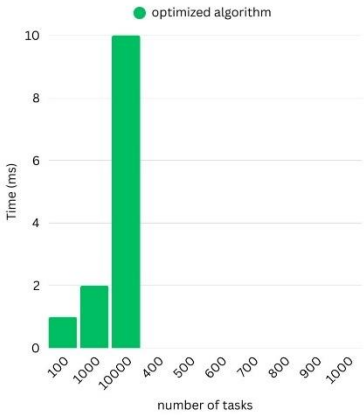
3.3 Performance Analysis:

To evaluate and compare the actual performance for both naive and optimized algorithms, a series of tests were conducted using datasets of varying sizes. The following result summarizes the execution times using different input sizes. Below are the results and efficiency improvements observed for each algorithm.

Sample size (Words)	Naive (ms)	Optimized (ms)	Results
Small – 100	1ms	1ms	0%
Medium – 1k	2ms	2ms	0%
Large – 10k	67ms	10ms	85%

The results clearly demonstrate that the optimized algorithm is significantly faster than the naive algorithm, especially as the input size increases.





Conclusion

4.1 Comparison

The naive algorithm is simpler and uses less memory, but it is slower with large datasets.

The optimized algorithm improves performance significantly through in-memory processing and structured insertion.

4.2 Final Thoughts

Efficient scheduling requires balancing simplicity and performance. The optimized SCIN+ algorithm offers a scalable and effective solution for real-world time management systems.

Reference:

Miqdaid Osama, Technical Decomposition and Performance Analysis of the Task scheduling algorithm: Empirical Data and Analysis.

Merge sort:

Introduction to Algorithms, fourth edition Hardcover – 5 April 2022

by Thomas H. Cormen chapter 4

programming language:

C++